

Algoritmos y Estructuras de Datos III

Examen parcial

1. **Dividir y conquistar.** Implemente una función `buscarMejor(L, f)` que dada una lista `L` y una función `f` permite encontrar el mejor entre dos números usando la estrategia de dividir y conquistar.

La función `f` no es necesario definirla, simplemente asumimos que dados dos valores, devuelve el mejor de ellos. Por ejemplo,

$$a = f(x, y)$$

hace que `a` sea el mejor valor entre `x` e `y` respecto de `f`.

2. **Listas 1.** Considere la clase `SingleLinkedList` que implementa una lista enlazada simple. Implemente el método:

```
eliminarUltimo(self) # que elimina el último nodo
                    # de la lista en caso de existir.
```

3. **Listas 2.** Considere la lista `DoubleLinkedList` que implementa una lista doblemente enlazada. Implemente el método:

```
insertarElemento(self, x, i) #que inserta el elemento x en la
posición i
```

4. **Queue.** Implemente las clases `Node` y `Stack` utilizando una lista doblemente enlazada.

Para la clase `Node`: implemente el método `__init__`

Para la clase `Stack`: implemente el método `__init__`, `enqueue`, `dequeue` e `invertir`. El método `invertir` altera el orden por el que se insertan y retiran elementos de la lista. De modo que si se inserta un elemento `x` y luego se invierte la `Queue`, el próximo elemento a retirar será `x`.